

# Abstracting Type in C

Thomas

March 23, 2025

## Contents

|          |                                                                                            |          |
|----------|--------------------------------------------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                                                        | <b>2</b> |
| <b>2</b> | <b>Namespaces in C</b>                                                                     | <b>3</b> |
| 2.1      | The Middle Dot Convention . . . . .                                                        | 3        |
| 2.2      | Benefits of This Approach . . . . .                                                        | 3        |
| 2.3      | Implementation Details . . . . .                                                           | 3        |
| 2.4      | Comparison with C++ Namespaces . . . . .                                                   | 4        |
| <b>3</b> | <b>Tableau and FG Tables</b>                                                               | <b>4</b> |
| 3.1      | Tableau . . . . .                                                                          | 4        |
| 3.2      | Function Given (FG) Tables . . . . .                                                       | 5        |
| 3.3      | The Relationship Between Tableaux and FG Tables . . . . .                                  | 5        |
| <b>4</b> | <b>Implementing Templates in C</b>                                                         | <b>6</b> |
| 4.1      | The Xi () Macro System . . . . .                                                           | 6        |
| 4.1.1    | Implementation Details . . . . .                                                           | 6        |
| 4.2      | CVT: Cell Value Type . . . . .                                                             | 6        |
| 4.3      | Template-Based Type Generation . . . . .                                                   | 7        |
| 4.4      | Function Given (FG) Tables . . . . .                                                       | 7        |
| 4.5      | Benefits of This Approach . . . . .                                                        | 7        |
| 4.6      | Limitations . . . . .                                                                      | 8        |
| <b>5</b> | <b>Generalizing type</b>                                                                   | <b>8</b> |
| 5.1      | Passing a TM FG table instance . . . . .                                                   | 8        |
| 5.2      | Passing TM instance data, compile in the fg table . . . . .                                | 9        |
| 5.3      | Passing in both the TM instance data and corresponding TM<br>FG tables instances . . . . . | 9        |
| 5.4      | Using a "vtable" . . . . .                                                                 | 10       |

|          |                                                                 |           |
|----------|-----------------------------------------------------------------|-----------|
| 5.5      | Using a pointer pair . . . . .                                  | 10        |
| 5.6      | Instance curried dispatch function . . . . .                    | 10        |
| 5.7      | Self-typed Instance and dispatch function . . . . .             | 11        |
| 5.8      | Which approach? . . . . .                                       | 11        |
| <b>6</b> | <b>Implementation Details</b>                                   | <b>12</b> |
| 6.1      | Main objects . . . . .                                          | 12        |
| 6.2      | Declaration . . . . .                                           | 12        |
| 6.3      | Instantiation . . . . .                                         | 12        |
| 6.4      | Use . . . . .                                                   | 12        |
| 6.5      | FG table instances . . . . .                                    | 13        |
| 6.6      | One for all FG table . . . . .                                  | 13        |
| 6.7      | Separate FG tables . . . . .                                    | 14        |
| 6.8      | Direct access through instance field (Orrin suggestion) . . . . | 14        |
| <b>7</b> | <b>Observations</b>                                             | <b>15</b> |
| <b>8</b> | <b>Conclusion</b>                                               | <b>16</b> |

# 1 Introduction

Type abstraction is a fundamental challenge in C programming. Unlike object-oriented languages that provide built-in mechanisms for polymorphism, C requires explicit design patterns to achieve similar flexibility. This document explores various approaches to implementing type abstraction in C, using a tape machine (TM) as a concrete example.

The core problem we’re addressing is how to write generic code that can operate on different implementations of the same abstract interface. Specifically, we want to create functions that can work with any tape machine implementation without knowing the implementation details. This separation of interface from implementation facilitates creating modular, maintainable code in C.

Throughout this document, we use the notation (TM, CVT) to represent a generic type where TM is the tape machine and CVT is a template parameter for the cell value type. This allows us to discuss both implementation-agnostic and type-specific behaviors.

## 2 Namespaces in C

C does not have built-in namespace support like C++ or other modern languages. However, we can simulate namespaces using naming conventions. In this document, we use the middle dot character ( $\cdot$ ) as a namespace separator to organize identifiers into logical groups.

### 2.1 The Middle Dot Convention

The middle dot ( $\cdot$ ) in identifiers like ‘TM·Array’ or ‘TM·Function·fg’ is not a special operator in C. It is literally part of the identifier name. This convention creates a visual hierarchy that mimics namespaces while remaining valid C code.

Examples: “‘c / *TM is the primary namespace* / Array is a sub-namespace  
// init<sub>pe</sub> is a function in that namespace (TM·Array, CVT)·init<sub>pe</sub>  
/ *Core is the primary namespace* / Guard is a sub-namespace // check is  
a function in that namespace Core·Guard·fg.check(&chk, 1, tm, TM·Msg·tm)  
“

### 2.2 Benefits of This Approach

1. **Organization:** Related functions and types are visually grouped together
2. **Collision avoidance:** Reduces the risk of name collisions in large codebases
3. **Readability:** Makes it clear which component a function or type belongs to
4. **Consistency:** Provides a uniform naming scheme across the codebase

### 2.3 Implementation Details

Since the middle dot is part of the identifier, the C compiler treats these names as atomic units. For example, ‘TM·Array’ is a single identifier to the compiler, not two identifiers separated by an operator.

The macro system we discussed earlier helps generate these namespace-like identifiers consistently:

```
“‘c // Generates TM·int (TM, int)
// Generates TM·Array·int·initpe (TM·Array, int)·initpe “
```

## 2.4 Comparison with C++ Namespaces

Unlike C++ namespaces, our convention:

1. Does not provide actual scope isolation
2. Cannot be opened with ‘using’ directives
3. Is purely a naming convention, not a language feature

Despite these limitations, this convention provides many of the organizational benefits of true namespaces while remaining compatible with standard C.

## 3 Tableau and FG Tables

Two key concepts in our type abstraction approach are the "Tableau" and "Function Given (FG) Table." Understanding these concepts is essential for following the rest of this document.

### 3.1 Tableau

A "Tableau" is our term for a data structure that holds the state of a particular instance. In the context of our tape machine example:

1. **Definition:** A tableau is a struct that contains all the state variables needed for a specific implementation of an abstract interface.
2. **Purpose:** The tableau serves as a shared workspace where functions can read and write state information. Think of it as a blackboard where functions can leave "notes" for each other.
3. **Implementation-specific:** Each implementation of an abstract interface will have its own tableau structure with different fields appropriate to that implementation.

For example, the tableau for an array-based tape machine might look like:

```
“c struct { CVT *hd; / Current head position CVT *position; / Base  
position (leftmost cell) (extentt, CVT) extent; // Number of cells in the tape  
} (TM·Array, CVT); “
```

While a function-based implementation might have a completely different tableau structure.

### 3.2 Function Given (FG) Tables

The "Function Given" (FG) table is a collection of function pointers that implement operations on a specific tableau type.

1. **Definition:** An FG table is a struct containing function pointers that all take a tableau as their first argument (hence "Function Given").
2. **Purpose:** FG tables allow for polymorphic behavior in C by providing different implementations of the same interface.
3. **Structure:** Each function in the FG table takes a pointer to a tableau as its first argument, allowing it to access and modify the state.

A simplified example of an FG table for our tape machine might look like:

```
“c typedef struct { TM·Tape·Topo (*Tape·topo)(TM *tm); bool (*Tape·bounded)(TM *tm); TM·Head·Status (*Head·status)(TM *tm); Core·Status (*mount)(TM *tm); void (*step)(TM *tm); void (*rewind)(TM *tm); // ... other functions } TM·FG; “
```

When specialized for a specific cell value type (CVT), we might have:

```
“c typedef struct { TM·FG *fg; // Base function table (extentt, CVT) (*extent)(TM *tm); CVT (*read)(TM *tm); void (*write)(TM *tm, CVT *remotept); } (TM, CVT)·FG; “
```

### 3.3 The Relationship Between Tableaux and FG Tables

The tableau and FG table work together to implement polymorphism:

1. Each implementation provides its own tableau type and corresponding FG table.
2. The first field of a tableau is typically a pointer to its corresponding FG table.
3. Functions in the FG table operate on the tableau, reading and writing its state.
4. Client code can work with any implementation by using the functions in the FG table without knowing the details of the tableau structure.

This pattern allows us to achieve polymorphic behavior in C while maintaining type safety and avoiding the overhead of dynamic dispatch mechanisms like those used in object-oriented languages.

## 4 Implementing Templates in C

C lacks built-in support for templates or generics, unlike C++ or modern languages like Java or Rust. However, we can implement template-like functionality using the C preprocessor. This section explains the template system used throughout this document.

### 4.1 The Xi ( ) Macro System

We use a special macro system, represented by the Greek letter Xi ( ), to create template-like behavior in C. This system allows us to parameterize types and functions, similar to templates in C++.

The `xi` macro concatenates identifiers with a middle dot (·) separator:

```
““c (TM, CVT) -> TM·CVT (TM·Array, CVT)·init -> TM·Array·CVT·init  
““
```

This allows us to create type and function names that are specialized for specific types, while maintaining a consistent naming convention.

#### 4.1.1 Implementation Details

The `xi` macro system is implemented using variadic macros and the C preprocessor's token concatenation operator (`##`):

```
““c #define _2(a, b) a##_##b #define 2(a, b) _2(a, b) ““
```

The system includes macros for different numbers of arguments (0 through 10) and uses argument counting to automatically select the appropriate macro:

```
““c #define (...) EXPAND(COUNT_ARGS(VA_ARGS), VA_ARGS) ““
```

This allows for flexible usage with different numbers of template parameters.

### 4.2 CVT: Cell Value Type

Throughout our tape machine implementation, we use CVT (Cell Value Type) as a template parameter. This represents the type of data stored in each cell of the tape.

The implementation uses conditional compilation to handle different CVT values:

```
““c #ifndef CVT // Code for generic, non-specialized case #endif  
#ifdef CVT // Code specialized for a specific CVT #endif ““
```

When using the library, you include it multiple times with different CVT definitions:

```

““c // First include with CVT undefined for base definitions #include
"TM.lib.c"
// Then include with CVT defined for each type specialization #define
CVT int #include "TM.lib.c" #undef CVT
#define CVT float #include "TM.lib.c" #undef CVT ““

```

### 4.3 Template-Based Type Generation

The template system generates several types of identifiers:

1. **Type names:** ‘(TM, CVT)’ becomes ‘TM·int’ when CVT is defined as ‘int’
2. **Function names:** ‘(TM·Array, CVT)·init<sub>pe</sub>’ becomes ‘TM·Array·int·init<sub>pe</sub>’
3. **Variable names:** ‘(extent<sub>t</sub>, CVT)’ becomes ‘extent<sub>t</sub>·int’

### 4.4 Function Given (FG) Tables

A key part of our implementation is the Function Given (FG) table pattern. Each implementation of the tape machine provides an FG table containing function pointers for its operations.

The FG tables are specialized based on the CVT:

```

““c typedef struct { TM·FG *fg; // Base function table (extentt, CVT)
(*extent)(TM *tm); CVT (*read)(TM *tm); void (*write)(TM *tm, CVT
*remotept); } (TM, CVT)·FG; ““

```

This allows for type-safe operations while maintaining a consistent interface.

### 4.5 Benefits of This Approach

1. **Type safety:** The compiler checks that the correct types are used with each specialized function
2. **Code reuse:** Common functionality is defined once and specialized for different types
3. **Consistent naming:** The macro ensures a consistent naming convention
4. **Modularity:** Different implementations can share the same interface

## 4.6 Limitations

1. **Verbosity:** More verbose than native templates in C++
2. **Complexity:** Requires understanding of preprocessor macros and conditional compilation
3. **Debugging:** Preprocessor-generated code can be harder to debug
4. **Limited IDE support:** Most IDEs don't understand this template system for code completion

Despite these limitations, this approach provides a powerful way to implement generic, type-safe code in C without resorting to void pointers or code duplication.

## 5 Generalizing type

### 5.1 Passing a TM FG table instance

1. Suppose we have a generic TM FG table with the declared functions constituting the architecture (user's view, interface) of a tape machine.
2. Instances of this TM FG table represent different implementations of the architecture. For example, `TM·Array·fg` for a TM implemented as an array, and `TM·Function·fg` for a function implementation (similar to the `NaturalNumber` in the Java implementation).
3. Now suppose we have a function 'map' that is given a `TMread`, a `TMwrite`, and a function. The map does not care how the TM is implemented.

If these first two arguments are declared as TM FG type, then they can accept any fg table whether it be an Array implementation, a Function implementation, etc. The map can then pick the functions it needs, such as `step` and `rightmost`, from the table, and thus get tape behavior.

This achieves what we wanted: a map function that can use a tape machine and does not care what the implementation of that tape machine is.

However, the TM functions that map calls require a first argument that holds the TM instance data. Without this, it cannot know which machine is being operated on. Furthermore, the instance data type must match the TM fg table type.

Given steps 1 through 3, we never passed in instance data to map, only a TM FG table instance. So map has a type, but not the data.

## 5.2 Passing TM instance data, compile in the fg table

1. As noted, ‘map’ is given a  $TM_{read}$ , a  $TM_{write}$ , and a function. For these first two arguments, suppose we pass in the instance data, i.e., struct pointers.
2. The TM-Array struct will hold a base pointer to the array and the head location. The TM-Function struct would hold a handle recognized by the functions or some state used by them.
3. Then we have a map function for each architecture type, and we compile into each of these a call to the correct default TM-fg table. The map-Array will have TM-Array-fg compiled into it, and the map-Function will have TM-Function-fg compiled into it, etc.
4. This approach works, but we duplicate the map code many times just to get the types to match the calls.

## 5.3 Passing in both the TM instance data and corresponding TM FG tables instances

1. Say we have one generic ‘map’ that takes 5 arguments:  $TM_{read}$  (an instance),  $TM_{readfg}$ , a  $TM_{write}$  (an instance), and  $TM_{writefg}$ , plus the function to apply.
2. The instance parameters must be declared ‘void \*’ to prevent argument checking on them; otherwise, it will be necessary to implement many map functions for each instance type combination.
3. This works, though we have no way to know if the correct type of instance data that matches the fg table was actually passed in.
4. We could create many wrapper functions to accomplish the instance data type check, then call a central map function with ‘void \*’ instance data pointers. With any luck, the optimizer would make this extra layer go away, so it will be efficient. Still, map now has 5 arguments instead of the three that are natural for the problem.

## 5.4 Using a "vtable"

1. All TM instance types are structs with a first field that points to the fg table, regardless of the implementation type. This is not a serious requirement because each implementation type already has a struct for its implementation data. However, it does make an instance bigger, and it is not clear that the optimizer will be able to optimize it effectively.
2. Then a generic fg table is created, where the functions have the signatures as specified in the FG table. However, each of these functions follows the fg table pointer in the instance data given to it as a first argument, and then calls that function.
3. It works, and it is a tried and true method.

## 5.5 Using a pointer pair

1. Instead of the instance data being called the 'TM type', a pair is the 'TM type'. The first member of the pair points to an instance, the second to a corresponding fg table.
2. Again there are some generic TM functions, each accepts a pair as a first argument, and then calls the corresponding fg table function while giving it the instance pointer from the pair as the first argument.
3. This approach packs the same information as the vtable approach. If there is a one-to-one correspondence between instances and pairs, then it is larger by one pointer.
4. This is a type of 'link' from our continuations, but the tableau pointer is gone due to using a stack frame, and the continuation table pointer is gone due to flow-through sequencing. Thus it is two pointers instead of 4.
5. Hopefully, the optimizer would recognize that the pair pointers are redundant. They are only needed to ensure that the instance and fg table go together.

## 5.6 Instance curried dispatch function

1. In this case, a pointer to a function is the 'TM type'. There is an enum that gives numeric tags to each of the functions in the FG struct instance.

2. The tag for the desired fg function to be called is passed as the first argument to the dispatch function, and the remaining arguments are for the fg function.
3. The dispatch function has the instance data and fg table pointer curried into it, so it then calls the correct fg function while providing it with the correct instance data.
4. In C, it is not immediately obvious how to create a function on the fly each time an instance is created to curry the instance data into it, and then later how to call the newly created function.

### 5.7 Self-typed Instance and dispatch function

1. With this method, an extra field is added to the instance data that identifies its type. Then a globally available TM-specific dispatch function is called to run the desired fg function on the data. It is similar to the dispatch function from above, but the instance data is an additional argument.
2. The dispatch function uses the type information to know which fg table to call. This type information could be a pointer to the appropriate fg table.
3. This approach differs from the vtable method in that all the arguments are given to the one TM dispatch function, and it figures out how to do the dispatch. There is not a wrapper-per-function.

### 5.8 Which approach?

We want to be able to declare an instance as a local variable, where it will end up being on the stack frame. We want to avoid heap allocation for performance reasons, but not exclude the use of heap memory.

Dynamically created functions are out of the question. Dispatch approaches require the enum table to have tags. This is not a show-stopper. However, having one dispatch function means the argument types are not checked. That is a problem.

Doubled-up arguments (one for the instance, the other for the table) either require wrappers or do not check that an instance really goes with the data. Besides, it requires a lot of typing.

The pointer pair approach requires the separate maintenance of the pair. When these are local variables, the pair has to be tied to the instance. This

would require the user to be aware of the existence of both the instance data and the pair, to pass them to an initializer. This could be avoided if the pair were combined with the instance, but that is identical to the vtable approach (the instance pointer no longer being needed).

So we are led to the vtable approach, though we are still hoping for the curried dispatch function - lol.

## 6 Implementation Details

### 6.1 Main objects

1. (TM, CVT)·FG table (CVT is a template for the tape cell value type).
2. (TM, CVT) instance, typically declared as a local variable.
3. FG (TM·Array, CVT)·fg, which is a (TM, CVT)·FG type, provides pointers to function wrappers that are given a (TM, CVT) instance. Each wrapper then calls the function found in the instance's vtable.

### 6.2 Declaration

1. With CVT not defined, `#include "TM.lib.c"` in FACE section.
2. With each different CVT value defined, `#include "TM.lib.c"` in FACE section again.
3. Do the same in the IMPLEMENTATION section.

### 6.3 Instantiation

1. Define a local variable, or get from malloc, a (TM, CVT) type, say 'tm'.
2. Call `init(tm)`.

### 6.4 Use

1. Call, for example, `(TM·Array, CVT)·fg.step(tm)`, where tm is a (TM, CVT) type.

## 6.5 FG table instances

There can be multiple instances of an FG table, each with functions that are implemented differently, though they share the same FG table type.

Each CVT and FG table instance type will have a different tableau struct. Each tableau struct will have the type  $(TM \cdot \langle type \rangle, CVT)$ , where  $\langle type \rangle$  is the FG instance table type. For example,  $(TM \cdot \text{Array}, CVT)$  is a struct of state variables for an Array implementation of the TM.

## 6.6 One for all FG table

The "TM.lib.c" file defines an FG type called  $(TM, CVT) \cdot FG$ . Note that CVT is a template parameter. Hence, giving different CVT bindings, there will be different FG tables.

For each  $(TM, CVT) \cdot FG$ , there can be a number of  $(TM \cdot \langle implementation \rangle, CVT) \cdot fg$  instances. For example,  $(TM \cdot \text{Array}, CVT) \cdot fg$ . Each of these is a parameterized table.

A user has an fg table to call functions from. This is either passed in or, more likely, found at global scope after including "TM.lib.c", and then including it again for each value of the CVT variable to be used.

For each implementation type, there is a struct defined called  $(TM \cdot \langle implementation \rangle, CVT)$ , note the lack of further suffix. Instances of this type are said to be instances of the tape machine. 'FG' stands for 'function given', and what each said function is given is one or more arguments of the  $(TM \cdot \langle implementation \rangle, CVT)$  type.

When a user wants a tape machine, the user makes an instance of a  $(TM \cdot \langle implementation \rangle, CVT)$ . This is then handed to a function called  $(TM \cdot \langle implementation \rangle, CVT) \cdot \text{init}()$ . The init function is given whatever parameters are needed for initializing the instance, and it returns a more generalized pointer of type  $(TM, CVT) *$ .

After initialization, the  $(TM \cdot \langle implementation \rangle, CVT)$  instance will have a first field value that points to the  $(TM \cdot \langle implementation \rangle, CVT) \cdot fg$  table. However, the pointer to this instance will be of the type returned by init, which is  $(TM, CVT) *$  (notice it lost the implementation). This more generic pointer can now be used with the generic instance of the  $(TM, CVT) \cdot FG$  table,  $(TM, CVT) \cdot fg$ , which holds wrapper functions that make use of the first field value to find the real fg table and to call those functions.

Hence, after the user calls the correct init for the specific instance made, the resulting pointer points to the same type object independent of the implementation of the functions. Thus, code written to manipulate TMs can

be written in a manner that is agnostic to the implementation of those TMs.

When a function is called, say  $(TM, CVT) \cdot fg.step(tm)$ , it is passed the generic type  $'(TM, CVT) *$ ,  $tm$ . This will call the step wrapper function, which will in turn pass the same arguments (in this case, only  $'tm'$ ) to the step function found in the  $fg$  table pointed to by the first member of the  $tm$  struct.

## 6.7 Separate FG tables

In the flat  $fg$  table approach described in the prior section, each wrapper function table of the form  $(TM, CVT) \cdot fg$ , obtained by setting the CVT template to a type value, will have a full set of wrapper function pointers. When a wrapper function is the same independent of the CVT variable, it still has a pointer copied into each  $fg$  table. Thus, there will be a lot of redundancy in the tables if many functions are not CVT-specific.

For the layered  $fg$  table, there are separate wrapper function tables for the wrappers that are independent of CVT and those that are CVT-differentiated. The user then calls functions from the CVT-independent table when invoking those functions and from the CVT-differentiated functions table when calling those. So, for example, the user would call  $TM \cdot fg.step(tm)$  to do a step, as that is CVT-independent, but call  $(TM, CVT) \cdot fg.read(tm)$  to do a read, as the read value is CVT-differentiated.

## 6.8 Direct access through instance field (Orrin suggestion)

Instead of calling  $fg$  functions through a globally visible table such as:

```
““c fg->step(tm); // Wrapper style ““
```

where  $fg$  is of type  $(TM, CVT) \cdot FG$  and calls to it require globally accessible wrapper functions,

we can use the instance's first field directly:

```
““c tm->fg->step(tm); ““
```

Here,  $tm$  is a  $(TM, CVT) *$  — the generic type returned by `init` — and  $fg$  is the first field of the instance pointing to the correct  $FG$  table.

This style removes the need for wrapper functions entirely. Each TM implementation embeds its corresponding  $FG$  table pointer directly in the instance. Thus, no global wrapper needs to be referenced, and function type-checking is preserved at compile time.

If there are CVT-independent functions, we can use a layered  $FG$  table. Each CVT-specific  $FG$  table includes a pointer to a base table:

“c tm->fg->base->step(tm); / *Access shared, CVT-independent function tm->fg->read(tm); / Access CVT-specific function* “

This structure mimics prototype delegation in JavaScript, except we resolve the chain explicitly with types, not dynamic name resolution.

The benefit of this approach is that the user only needs to have the instance in scope — no global FG table declarations. All function dispatch is local and type-safe.

The only remaining user burden is knowing whether a function is CVT-specific or not. This is made obvious by whether the function appears in the base or CVT-differentiated FG struct, and the compiler will catch mismatches.

(Contribution by Orrin — co-author)

## 7 Observations

C does not have automatic conversion of a child type to its parent; instead, it will give a type mismatch error.

The vtable pattern with separate tables will have a CVT-differentiated fg table pointer on the tableau. Without the function wrappers, a call can occur like this:

“c tm->fg->step(tm) “

Or even with the wrappers and a globally available fg table:

“c fg->step(tm) “

In both of these, ‘step’ will be the CVT-differentiated version of step, so types will match, and these calls work.

However, things change when we want to call the non-CVT-differentiated shared table:

“c tm->TM.fg->mount(tm) “

Due to ‘mount’ being in the shared fg table that has type ‘TM.fg’ where it expects to be given arguments of type ‘TM \*’, and having type ‘TM.<CVT>’, there will be a type mismatch error on the call. The programmer will be compelled to fix this with a cast:

“c tm->TM.fg->mount((TM \*)tm) “

Requiring the programmer to defeat the type system by habit is not a good approach.

So the one solution that does work is the one big table per type. All functions will then have correct signatures in the FG declaration. In the implementation, the more generic functions can be cast when assigned directly

to the fg table function pointers, or wrappers that delegate can be written and assigned to the fg table function pointers.

## 8 Conclusion

After examining various approaches to type abstraction in C, the vtable pattern emerges as the most practical solution for our tape machine implementation. While it has some overhead in terms of memory usage, it provides the best balance of type safety, ease of use, and performance.

The direct access through instance field approach suggested by Orrin offers an elegant alternative that eliminates the need for wrapper functions while maintaining type safety. This approach is particularly appealing for its simplicity and the fact that it keeps all the necessary information localized to the instance.

For projects where performance is critical, the "one big table per type" approach may be the most efficient, as it avoids the need for type casting while still providing a clean interface.

Ultimately, the choice of approach depends on the specific requirements of the project, but the vtable pattern and its variations provide a solid foundation for type abstraction in C programming.